Отчёт по лабораторной работе 8

XOR гаммирование и повторное использование ключа

Архипов Александр Сергеевич

12.10.2026

Реализовано XOR-шифрование байтов UTF-8 с ключом из secrets.token_bytes. Проверено восстановление двух сообщений и показана учебная атака при повторном использовании одного ключа.

Содержание

1 Введение **6**
1.1 Актуальность 6
1.2 Цель работы 6
1.3 Задачи 6
1.4 Объект и предмет исследования 7
1.5 Методы исследования 7
1.6 Схема выполнения работы 7

2 Теоретическая часть **9**

3 Материалы и методы **10**

4 Выполнение лабораторной работы **11**
4.1 Атака на повторно использованный XOR-ключ 11
4.2 Точное XOR-шифрование и расшифровка UTF-8 12

5 Результаты **14**

6 Обсуждение результатов **15**

7 Заключение **16**

Список литературы **17**

A Листинг программы **18**

Список иллюстраций

1.1 Схема выполнения лабораторной работы 8

4.1 Атака на повторно использованный XOR-ключ 12
4.2 Точное XOR-шифрование и расшифровка UTF-8 13

Список таблиц

5.1 Результаты XOR-преобразования и учебной атаки . . . 14

1 Введение

Исполнитель — Архипов Александр Сергеевич, группа НБИбд-01-24. Студенческий билет 1132239106. Электронная почта: 1132239106@rudn.ru.

1.1 Актуальность

Повторное использование гаммы связывает шифротексты разных сообщений. Учебный пример позволяет проследить, как известный открытый текст раскрывает другое сообщение при совпадающем ключе.

1.2 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

1.3 Задачи

1. Преобразовать две строки в байты UTF-8 и проверить равенство длин.

2. Сформировать случайный ключ длиной 25 байт.
3. Реализовать XOR-шифрование и расшифрование.
4. Восстановить второе сообщение по C1, C2 и известному P1 при повторении ключа.
5. Проверить восстановление ключа и исходных сообщений.

1.4 Объект и предмет исследования

Объект исследования — байтовые представления строк КодоваяФраза1 и Безопасность2. Предмет исследования — обратимость XOR и утечка при повторении ключа.

1.5 Методы исследования

Использованы побайтовое XOR-преобразование, генерация ключа через secrets.token_bytes и автоматические проверки assert для расшифрования и учебной атаки.

1.6 Схема выполнения работы

Последовательность действий показана на рис. 1.1.



Рисунок 1.1: Схема выполнения лабораторной работы

2 Теоретическая часть

XOR применяется к каждой паре байтов одинаковой длины. Основные свойства операции — a XOR a = 0 и a XOR 0 = a — обеспечивают обратимость:

$$C = P \oplus K, \qquad P = C \oplus K.$$

При повторении ключа выполняется:

$$C_1 \oplus C_2 = P_1 \oplus P_2,$$

$$P_2 = C_1 \oplus C_2 \oplus P_1, \qquad K = C_1 \oplus P_1.$$

Эти равенства позволяют восстановить позиции второго сообщения, для которых известны соответствующие байты первого. Они не означают автоматического восстановления двух произвольных неизвестных сообщений.

Функция secrets.token_bytes возвращает случайную последовательность заданной длины, используя источник случайности операционной системы [1]. Случайность ключа не устраняет уязвимость его повторного использования.

3 Материалы и методы

Программа lab08.py запущена в консольной среде Ubuntu. Каждая строка занимает 25 байт UTF-8. Ключ генерируется заново при запуске; его повторение для двух сообщений используется внутри учебного примера. Программа имеет обычный режим и режим attack. Шифротексты разных запусков могут отличаться.

4 Выполнение лабораторной работы

4.1 Атака на повторно использованный XOR-ключ

При одном ключе C1 XOR C2 XOR P1 восстанавливает P2; C1 XOR P1 восстанавливает ключ. Обе проверки assert прошли. Шифротексты меняются при новом запуске.

Результат показан на рис. 4.1.

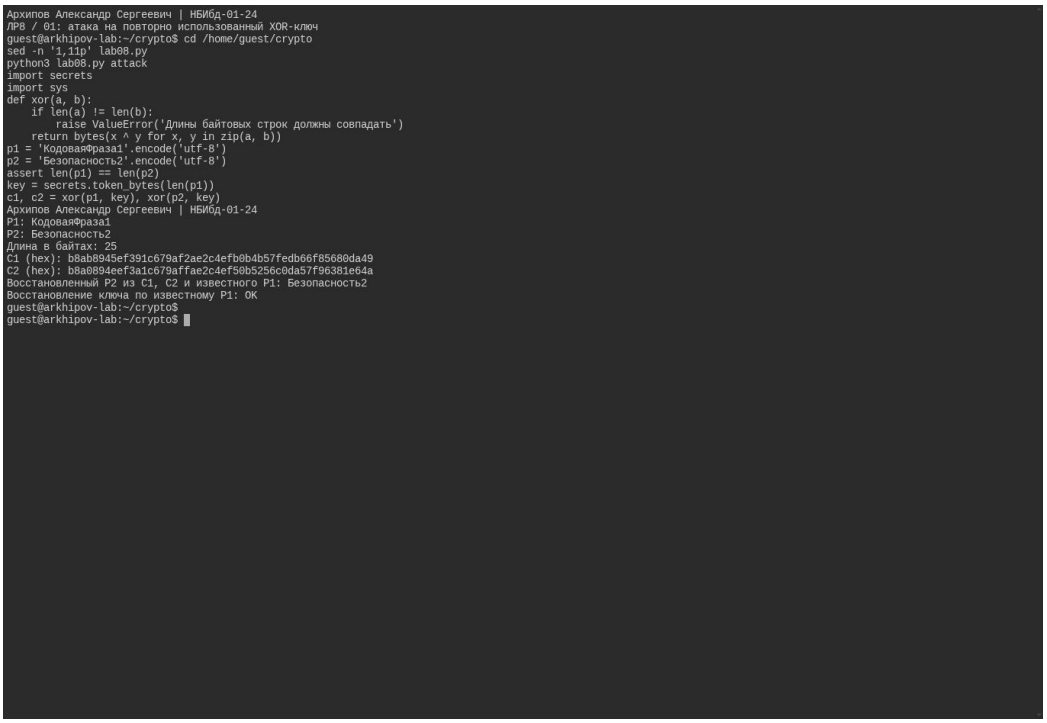


Рисунок 4.1: Атака на повторно использованный XOR-ключ

4.2 Точное XOR-шифрование и расшифровка UTF-8

XOR применяется к байтам UTF-8; обе фразы имеют длину 25 байт. Обратное преобразование восстанавливает каждую фразу точно, без замены букв. Здесь новый случайный ключ, отличный от снимка атаки.

Результат показан на рис. 4.2.

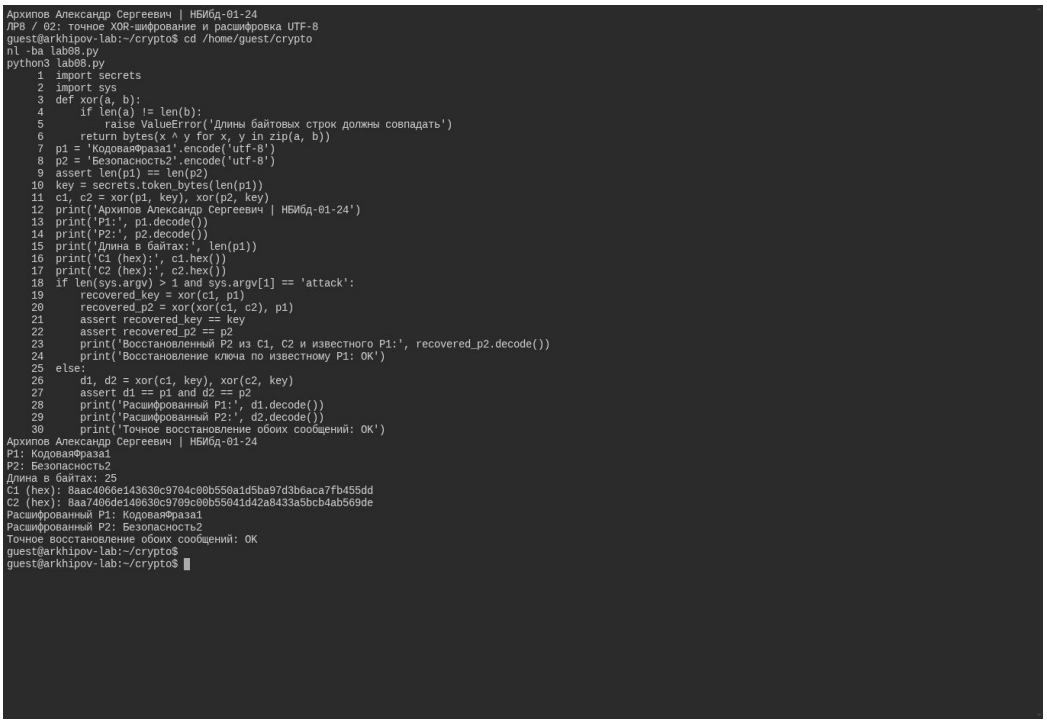


Рисунок 4.2: Точное XOR-шифрование и расшифровка UTF-8

5 Результаты

Таблица 5.1: Результаты XOR-преобразования и учебной атаки

Проверка	Результат
Размер P1 и P2	По 25 байт UTF-8
Обычный режим	Оба сообщения восстановлены точно
Ключ из C1 и известного P1	Совпадает с исходным ключом
P2 из C1 XOR C2 XOR P1	Получено Безопасность2
Автоматические проверки	Все условия assert выполнены

Снимки обычного режима и атаки относятся к разным запускам и используют разные случайные ключи. Сравниваются свойства восстановления, а не совпадение шифротекстов между снимками.

6 Обсуждение результатов

Точное восстановление строк подтверждает корректную работу с байтами UTF-8: ключ соответствует длине закодированного сообщения, а не количеству видимых символов.

Учебная атака показывает, что повторение ключа сокращает задачу до XOR известных величин. Условия одноразового гаммирования требуют не использовать одну гамму повторно; один только выбор криптографически подходящего генератора не обеспечивает это условие.

7 Заключение

- Реализовано обратимое XOR-преобразование сообщений UTF-8.
- Подтверждено восстановление второго сообщения и ключа при наличии известного первого сообщения.
- Показана необходимость исключать повторное использование одной гаммы.

Список литературы

1. Python Software Foundation. secrets Generate secure random numbers [Электронный ресурс]. URL: https://docs.python.org/3/library/secrets.html (дата обращения: 12.10.2026).

А Листинг программы

```
import secrets
import sys
def xor(a, b):
    if len(a) != len(b):
        raise ValueError('Длины байтовых строк должны совпадать')
    return bytes(x ^ y for x, y in zip(a, b))
p1 = 'КодоваяФраза1'.encode('utf-8')
p2 = 'Безопасность2'.encode('utf-8')
assert len(p1) == len(p2)
key = secrets.token_bytes(len(p1))
c1, c2 = xor(p1, key), xor(p2, key)
print('Архипов Александр Сергеевич | НБИбд-01-24')
print('P1:', p1.decode())
print('P2:', p2.decode())
print('Длина в байтах:', len(p1))
print('C1 (hex):', c1.hex())
print('C2 (hex):', c2.hex())
if len(sys.argv) > 1 and sys.argv[1] == 'attack':
    recovered_key = xor(c1, p1)
    recovered_p2 = xor(xor(c1, c2), p1)
    assert recovered_key == key
    assert recovered_p2 == p2
```

```
    print('Восстановленный P2 из C1, C2 и известного P1:',
     ↪  recovered_p2.decode())
    print('Восстановление ключа по известному P1: OK')
else:
    d1, d2 = xor(c1, key), xor(c2, key)
    assert d1 == p1 and d2 == p2
    print('Расшифрованный P1:', d1.decode())
    print('Расшифрованный P2:', d2.decode())
    print('Точное восстановление обоих сообщений: OK')
```